```
        {
            return std::unique_ptr<node>();
        }
        return pop_head();
    }

    std::unique_ptr<node> try_pop_head(T& value)
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);
        if(head.get()==get_tail())
        {
            return std::unique_ptr<node>();
        }
        value=std::move(*head->data);
        return pop_head();
     }

public:
    std::shared_ptr<T> try_pop()
    {
        std::unique_ptr<node> old_head=try_pop_head();
        return old_head?old_head->data:std::shared_ptr<T>();
    }

    bool try_pop(T& value)
    {
        std::unique_ptr<node> const old_head=try_pop_head(value);
        return old_head;
    }

    void empty()
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);
        return (head.get()==get_tail());
    }
};
```

This queue implementation will serve as the basis for the lock-free queue covered in chapter 7. It's an *unbounded* queue; threads can continue to push new values onto the queue as long as there's available memory, even if no values are removed. The alternative to an unbounded queue is a *bounded* queue, in which the maximum length of the queue is fixed when the queue is created. Once a bounded queue is full, attempts to push further elements onto the queue will either fail or block, until an element has been popped from the queue to make room. Bounded queues can be useful for ensuring an even spread of work when dividing work between threads based on tasks